Introduction to Modules in Python

Python modules are **pre-written code** that can be used in your programs to perform specific tasks. They are like **libraries** that contain a set of functions, classes, and variables that can be imported and used in your code.

Key Points

- **Modules** are reusable pieces of code that can be imported into your program.
- There are two types of modules: **built-in modules** and **external modules** .
- **Built-in modules** are pre-installed with Python and can be used directly, such as `math` and `random`.
- **External modules** are not pre-installed and need to be installed using **pip** , such as `pandas` and `scikit-learn`.
- **Pip** is a package manager that allows you to install and manage external modules.

How to Use Modules

1. Import a module using the `import` statement, such as `import pandas`.
2. Use the module's functions and classes in your code, such as `pd.read_csv()`.
3. Install external modules using pip, such as `pip install pandas`.

Key Takeaways

- Modules are a way to reuse code and make your programs more efficient.
- Built-in modules are pre-installed with Python, while external modules need to be installed using pip.
- Pip is a powerful tool for managing external modules and dependencies.
- Using modules can save you time and effort, and make your code more readable and maintainable.

Example Use Case

For example, if you want to read a CSV file, you can use the `pandas` module, which has a function called `read_csv()`. You can import the module and use the function like this: `import pandas as pd; df = pd.read_csv('file.csv')`. This is much easier than writing your own code to read a CSV file from scratch.